```
<!DOCTYPE html> <html lang="es"> <head> <meta charset="UTF-8"> <meta name="viewport"
content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
<title>Reloj Solar</title> <meta name="theme-color" content="#d97706"> <meta
name="description" content="Calculadora de Hora Solar Verdadera en tiempo real según
coordenadas geográficas."> <meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
<!-- Tailwind CSS --> <script src="https://cdn.tailwindcss.com"></script>
<!-- FontAwesome Icons --> <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
<style> @import
url('https://fonts.googleapis.com/css2?family=Cinzel:wght@500;700&family=JetBrains+Mono:w
ght@400;700&family=Plus+Jakarta+Sans:wght@400;600;800&display=swap');
body { font-family: 'Plus Jakarta Sans', sans-serif; touch-action: manipulation; }
.font-mono-custom { font-family: 'JetBrains Mono', monospace; }
.font-serif-custom { font-family: 'Cinzel', serif; }
/* Animación del pulso solar suave */ @keyframes sunGlow { 0%, 100% { filter: drop-shadow(0
0 15px rgba(245, 158, 11, 0.25)); } 50% { filter: drop-shadow(0 0 30px rgba(245, 158, 11, 0.45));
} }
.sun-glow { animation: sunGlow 6s ease-in-out infinite; }
/* Ocultar scrollbar en listas de sugerencias */ .no-scrollbar::-webkit-scrollbar { display: none; }
.no-scrollbar { -ms-overflow-style: none; scrollbar-width: none; } </style> </head> <body
class="bg-stone-950 text-stone-100 min-h-screen flex flex-col justify-between p-4 sm:p-6
select-none overflow-x-hidden">
<!-- CABECERA: Nombre de la ubicación actual --> <header class="w-full text-center pt-2
pb-4"> <div class="inline-flex items-center gap-2 px-3 py-1 rounded-full bg-stone-900/80 border
border-stone-800 mb-2"> <i class="fa-solid fa-location-dot text-amber-500 text-xs"></i> <span
id="locationType" class="text-[10px] tracking-widest uppercase font-bold
text-amber-500/90">Ubicación Seleccionada</span> </div> <h1 id="cityName" class="text-2xl
sm:text-3xl font-extrabold text-stone-100 tracking-tight leading-tight px-4 truncate"> Cargando...
</h1> <p id="countryName" class="text-sm font-medium text-stone-400 mt-1"> Buscando
coordenadas... </p> </header>
<!-- CENTRO: Reloj Solar Análogo y Lectura Digital Gigante --> <main class="flex-1 flex flex-col
items-center justify-center my-auto py-2">
<!-- Ícono del Sol Pastel/Mostaza con Manecillas Clásicas --> <div class="relative w-48 h-48
sm:w-56 sm:h-56 mb-6 flex items-center justify-center sun-glow"> <svg viewBox="0 0 200 200"
class="w-full h-full"> <!-- Rayos del Sol --> <g class="text-amber-400/80" stroke="currentColor"
stroke-width="4" stroke-linecap="round"> <line x1="100" y1="12" x2="100" y2="28" /> <line
x1="100" y1="172" x2="100" y2="188" /> <line x1="12" y1="100" x2="28" y2="100" /> <line
x1="172" y1="100" x2="188" y2="100" /> <line x1="38" y1="38" x2="49" y2="49" /> <line
x1="151" y1="151" x2="162" y2="162" /> <line x1="38" y1="162" x2="49" y2="151" /> <line
x1="151" y1="49" x2="162" y2="38" /> </g>
<!-- Círculo Base del Sol (Amarillo mostaza/suave) --> <circle cx="100" cy="100" r="62"
fill="#f59e0b" class="shadow-inner" /> <circle cx="100" cy="100" r="58" fill="#fbbf24" />
<!-- Marcadores de horas dentro del sol --> <circle cx="100" cy="48" r="2" fill="#1c1917" />
<circle cx="152" cy="100" r="2" fill="#1c1917" /> <circle cx="100" cy="152" r="2" fill="#1c1917"
/> <circle cx="48" cy="100" r="2" fill="#1c1917" />
<!-- Manecilla de Horas --> <line id="hourHand" x1="100" y1="100" x2="100" y2="65"
stroke="#1c1917" stroke-width="5.5" stroke-linecap="round" />
```

<!-- Manecilla de Minutos --> <line id="minuteHand" x1="100" y1="100" x2="100" y2="52" stroke="#1c1917" stroke-width="3.5" stroke-linecap="round" />
<!-- Puntero del Segundero --> <line id="secondHand" x1="100" y1="100" x2="100" y2="48" stroke="#78350f" stroke-width="1.5" stroke-linecap="round" />
<!-- Perno Central --> <circle cx="100" cy="100" r="5" fill="#1c1917" /> <circle cx="100" cy="100" r="2" fill="#fbbf24" /> </svg>
<!-- Lectura Digital Gigante de la Hora Solar -->   00:00:00   Hora Solar Verdadera (TST)
<!-- Comparativa secundaria con la Hora Oficial del Sistema -->   Hora oficial del dispositivo: 00:00:00
</main>
<!-- PIE DE PÁGINA: Búsqueda de Ciudad y Botón GPS --> <footer class="w-full max-w-md mx-auto pt-2 pb-2">
<!-- Campo de Búsqueda -->   <input type="text" id="citySearchInput" placeholder="Buscar país o ciudad..." autocomplete="off" class="w-full pl-10 pr-9 py-3 bg-stone-900 border border-stone-800 rounded-xl text-sm text-stone-100 placeholder-stone-500 focus:outline-none focus:border-amber-500 focus:ring-1 focus:ring-amber-500 transition-all shadow-lg" > <button id="clearSearchBtn" class="hidden absolute inset-y-0 right-0 pr-3 flex items-center text-stone-500 hover:text-stone-300"> </button>
<!-- Desplegable de Resultados -->
<!-- Botón Ubicación GPS --> <button id="gpsBtn" title="Usar mi ubicación GPS actual" class="p-3 bg-stone-900 border border-stone-800 rounded-xl text-amber-500 hover:bg-stone-800 active:scale-95 transition-all shadow-lg flex items-center justify-center shrink-0" > </button>
 </footer>
<!-- LÓGICA JAVASCRIPT & MOTOR PWA --> <script> // --- 1. BASE DE DATOS LOCAL OFFLINE DE RESPALDO --- const FALLBACK_CITIES = [{ name: "Santiago", country: "Chile", lat: -33.4489, lon: -70.6693 }, { name: "Buenos Aires", country: "Argentina", lat: -34.6037, lon: -58.3816 }, { name: "Madrid", country: "España", lat: 40.4168, lon: -3.7038 }, { name: "Ciudad de México", country: "México", lat: 19.4326, lon: -99.1332 }, { name: "Bogotá", country: "Colombia", lat: 4.7110, lon: -74.0721 }, { name: "Lima", country: "Perú", lat: -12.0463, lon: -77.0428 }, { name: "Sídney", country: "Australia", lat: -33.8688, lon: 151.2093 }, { name: "Tokio", country: "Japón", lat: 35.6762, lon: 139.6503 }, { name: "Londres", country: "Reino Unido", lat: 51.5074, lon: -0.1278 }, { name: "Nueva York", country: "Estados Unidos", lat: 40.7128, lon: -74.0060 }];
// Estado global de la ubicación let currentLocation = { name: "Santiago", country: "Chile", lat: -33.4489, lon: -70.6693, isGps: false };
// --- 2. CÁLCULO DE HORA SOLAR VERDADERA (TST) --- function getEquationOfTime(date) { const startOfYear = new Date(date.getUTCFullYear(), 0, 0); const diff = date - startOfYear; const dayOfYear = Math.floor(diff / (1000 * 60 * 60 * 24));
const B = (2 * Math.PI * (dayOfYear - 81)) / 365; const eot = 9.87 * Math.sin(2 * B) - 7.53 *

```
Math.cos(B) - 1.5 * Math.sin(B); return eot; // Retorna corrección en minutos }
function calculateTrueSolarTime(location, now) { const utcMinutes = (now.getUTCHours() * 60)
+ now.getUTCMinutes() + (now.getUTCSeconds() / 60) + (now.getUTCMilliseconds() / 60000);
// Ajuste de longitud: 4 minutos por cada grado geográfico const longitudeOffsetMinutes =
location.lon * 4;
// Corrección por Ecuación del Tiempo const eotMinutes = getEquationOfTime(now);
// Hora solar en minutos desde medianoche UTC let totalSolarMinutes = utcMinutes +
longitudeOffsetMinutes + eotMinutes;
// Normalizar en el rango de 24 horas (0 a 1440 minutos) totalSolarMinutes = (totalSolarMinutes
% 1440 + 1440) % 1440;
const hours = Math.floor(totalSolarMinutes / 60); const minutes = Math.floor(totalSolarMinutes
% 60); const seconds = Math.floor((totalSolarMinutes * 60) % 60);
return { hours, minutes, seconds, rawMinutes: totalSolarMinutes }; }
// --- 3. ACTUALIZACIÓN DE LA INTERFAZ DE USUARIO --- function updateClockDisplay() {
const now = new Date(); const solar = calculateTrueSolarTime(currentLocation, now);
// Formatear horas, minutos y segundos digitales const hStr = String(solar.hours).padStart(2,
'0'); const mStr = String(solar.minutes).padStart(2, '0'); const sStr =
String(solar.seconds).padStart(2, '0');
document.getElementById('solarTimeDisplay').textContent = hStr:{mStr}:${sStr};
// Actualizar hora del sistema const sysH = String(now.getHours()).padStart(2, '0'); const sysM =
String(now.getMinutes()).padStart(2, '0'); const sysS = String(now.getSeconds()).padStart(2, '0');
document.getElementById('systemTimeDisplay').textContent = sysH:{sysM}:${sysS};
// Rotación de las manecillas análogas sobre el sol const hourDeg = ((solar.hours % 12) +
solar.minutes / 60) * 30; const minDeg = (solar.minutes + solar.seconds / 60) * 6; const secDeg
= solar.seconds * 6;
document.getElementById('hourHand').setAttribute('transform', rotate(
hourDeg100100)); document.getElementById('minuteHand').setAttribute('transform', rotate(
{minDeg} 100 100)); document.getElementById('secondHand').setAttribute('transform',
rotate(${secDeg} 100 100)); }
function renderLocationHeader() { document.getElementById('cityName').textContent =
currentLocation.name; document.getElementById('countryName').textContent =
currentLocation.country; document.getElementById('locationType').textContent =
currentLocation.isGps ? 'Ubicación GPS Real' : 'Ubicación Manual'; }
function setLocation(loc) { currentLocation = loc; localStorage.setItem('reloj_solar_loc',
JSON.stringify(currentLocation)); renderLocationHeader(); updateClockDisplay(); }
// --- 4. BÚSQUEDA OPENSTREETMAP (NOMINATIM) & GPS --- let searchDebounceTimer;
const searchInput = document.getElementById('citySearchInput'); const searchResults =
document.getElementById('searchResults'); const clearSearchBtn =
document.getElementById('clearSearchBtn');
searchInput.addEventListener('input', (e) => { const query = e.target.value.trim();
clearTimeout(searchDebounceTimer);
if (query.length > 0) { clearSearchBtn.classList.remove('hidden'); } else {
clearSearchBtn.classList.add('hidden'); searchResults.classList.add('hidden'); return; }
if (query.length < 2) return;
searchDebounceTimer = setTimeout(() => { fetchCitiesFromNominatim(query); }, 350); });
clearSearchBtn.addEventListener('click', () => { searchInput.value = '';
clearSearchBtn.classList.add('hidden'); searchResults.classList.add('hidden'); });
async function fetchCitiesFromNominatim(query) { try { const response = await
```

```
fetch(https://nominatim.openstreetmap.org/search?format=json&q=${encodeURIComponent(qu
ery)}&addressdetails=1&limit=5&accept-language=es);
if (!response.ok) throw new Error("Error API"); const data = await response.json();
if (data && data.length > 0) { displaySearchResults(data.map(item => ({ name: item.address.city
|| item.address.town || item.address.village || item.display_name.split(',')[0], country:
item.address.country || '', lat: parseFloat(item.lat), lon: parseFloat(item.lon), isGps: false }))); }
else { fallbackLocalSearch(query); } } catch (err) { fallbackLocalSearch(query); } }
function fallbackLocalSearch(query) { const q = query.toLowerCase(); const matches =
FALLBACK_CITIES.filter(c => c.name.toLowerCase().includes(q) ||
c.country.toLowerCase().includes(q) ); displaySearchResults(matches); }
function displaySearchResults(results) { searchResults.innerHTML = ''; if (results.length === 0) {
searchResults.innerHTML = <div class="p-3 text-xs text-stone-500 text-center">Sin resultados
encontrados</div>; } else { results.forEach(res => { const item =
document.createElement('button'); item.className = 'w-full text-left px-4 py-3 border-b
border-stone-800/50 hover:bg-stone-800/80 transition-colors flex flex-col gap-0.5';
item.innerHTML = <span class="text-sm font-semibold text-stone-200">
res.name < /span >< spanclass = "text − xstext − stone − 400" >{res.country}</span>;
item.onclick = () => { setLocation(res); searchInput.value = '';
clearSearchBtn.classList.add('hidden'); searchResults.classList.add('hidden'); };
searchResults.appendChild(item); }); } searchResults.classList.remove('hidden'); }
// Botón GPS document.getElementById('gpsBtn').addEventListener('click', () => { if
('geolocation' in navigator) { navigator.geolocation.getCurrentPosition( async (pos) => { const lat
= pos.coords.latitude; const lon = pos.coords.longitude;
// Reversa simple para nombre o indicación general try { const res = await
fetch(https://nominatim.openstreetmap.org/reverse?format=json&lat=lat&lon =
{lon}&accept-language=es); const data = await res.json(); const name = data.address.city ||
data.address.town || data.address.village || "Mi Ubicación"; const country =
data.address.country || "GPS Real"; setLocation({ name, country, lat, lon, isGps: true }); } catch {
setLocation({ name: "Ubicación GPS", country: ${lat.toFixed(2)}°, ${lon.toFixed(2)}°, lat, lon,
isGps: true }); } }, (err) => { alert("No se pudo acceder a la geolocalización. Verifica los permisos
de tu dispositivo."); } ); } else { alert("Tu dispositivo no soporta geolocalización por GPS."); } });
// --- 5. REGISTRO PWA DINÁMICO --- function initPWA() { // Generar dinámicamente el
manifest de la PWA const manifest = { name: "Reloj Solar", short_name: "RelojSolar", start_url:
"./", display: "standalone", background_color: "#0c0a09", theme_color: "#d97706", icons: [{ src:
"data:image/svg+xml;utf8,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100
100'><circle cx='50' cy='50' r='40' fill='%23f59e0b'/><line x1='50' y1='50' x2='50' y2='25'
stroke='%231c1917' stroke-width='6' stroke-linecap='round'/><line x1='50' y1='50' x2='70'
y2='50' stroke='%231c1917' stroke-width='4' stroke-linecap='round'/></svg>", sizes: "512x512",
type: "image/svg+xml", purpose: "any maskable" }] };
const stringManifest = JSON.stringify(manifest); const blob = new Blob([stringManifest], {type:
'application/json'}); const manifestURL = URL.createObjectURL(blob);
const link = document.createElement('link'); link.rel = 'manifest'; link.href = manifestURL;
document.head.appendChild(link);
// Registrar Service Worker integrado if ('serviceWorker' in navigator) { const swCode =
self.addEventListener('install', (e) => self.skipWaiting()); self.addEventListener('activate', (e) =>
self.clients.claim()); self.addEventListener('fetch', (e) => e.respondWith(fetch(e.request).catch(()
=> caches.match(e.request))));; const swBlob = new Blob([swCode], { type:
'application/javascript' });
```

```
navigator.serviceWorker.register(URL.createObjectURL(swBlob)).catch(() => {}); } }
// --- 6. INICIALIZACIÓN DE LA APP --- window.addEventListener('DOMContentLoaded', () => {
// Cargar última ubicación guardada o predeterminada const saved =
localStorage.getItem('reloj_solar_loc'); if (saved) { try { currentLocation = JSON.parse(saved); }
catch (e) {} }
renderLocationHeader(); updateClockDisplay(); initPWA();
// Bucle de actualización cada segundo setInterval(updateClockDisplay, 1000); }); </script>
</body> </html>
```